INDEX

Name : Jaya Bharathi M.

Subject : MAD LAB Observation

Std : ..... Sec : ..... Roll No .....

School : .....

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1)	07/01/26	Build First Android Application Text Match Validator.	2	
2)	21/01/26	Implement Layout and Drawing Shapes in Android	4	
3)	28/01/26	App Navigation	6	
4)	04/02/26	Timer Application with Activity Lifestyle.	8	
5)	06/02/26	Implement UI Components using Jetpack	10	
6)	11/02/26	App Architecture - Persistence.		
7)	20/02/26	Advanced Recyclerview Implementation	12	
8)	25/02/26	Connect to the Internet	14	
9)	13/03/26	Respository Pattern and Work Manager	16	
10)	20/03/26	App UI Design using Navigation	18	

Completed

21/01/26

Experiment-2

Implement Layout and Drawing Shapes in Android.

AIM:

To design and implement an Android application using Kotlin and Jetpack compose that demonstrates layout management and allows the user to draw shapes such as line and a circle on the screen and control them using buttons.

ALGORITHM:

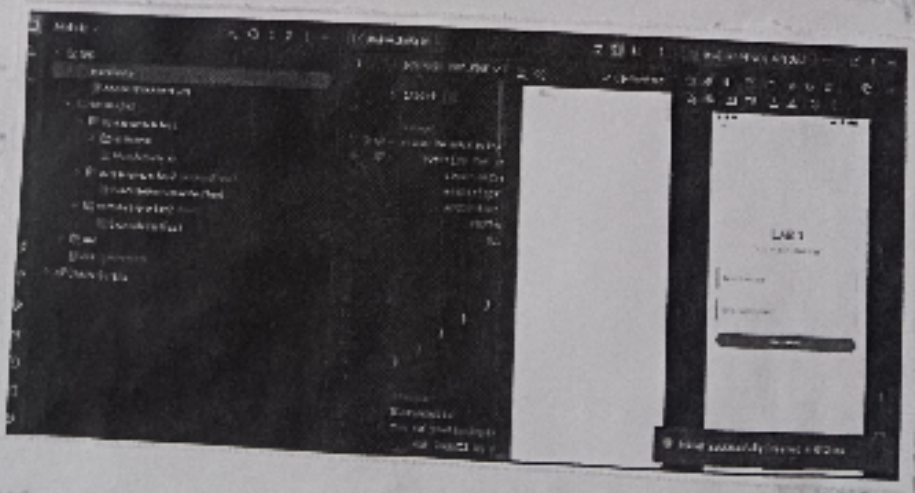
- Start the application.
- Open Android studio and create a new Android project using the Empty Activity template.
- Configure the project with the required Application name, package name, and minimum SDK version.
- Implement the MainActivity class to control the Application behaviours.
- Use Jetpack compose to design the user interface.
- Create column layout to arrange UI components vertically.
- Create a drawing area (Canvas or Box) where shapes will be displayed.
- Declare state variables to control whether a line or circle should be displayed.
- Add a Line button below the drawing area.
- When the line button is pressed:

  Update the state variable to display a horizontal line in the drawing area.

- Stores the values entered in both fields using state variables.
- When the button is clicked, retrieve the values from text fields.
- Compare the two input values using string comparison operation
- Show the result message on the screen or through a Toast / message component.
- Ensure the application runs properly on the Android Emulator or physical device.
- Stop the application.

Result:

A simple Android application was successfully developed using Kotlin and Jetpack Compose to compare two text inputs and display whether the entered texts match or not.



28/01/26 Experiment - 3

App Navigation.

AIM:

To develop an Android application using Kotlin and Jetpack Compose that demonstrates navigation between two screens using a button.

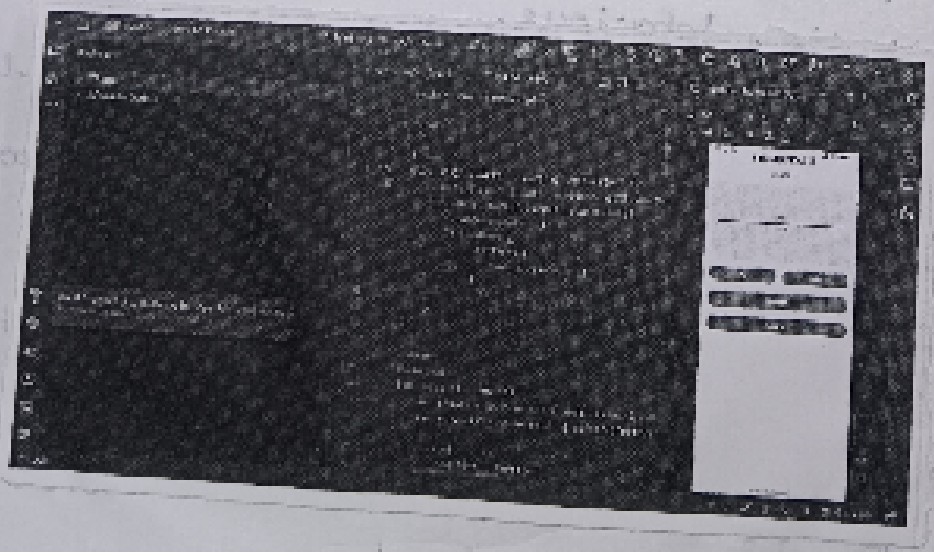
ALGORITHM:

- Start the application.
- Create a new Android project in Android Studio using Kotlin
- Implement MainActivity to display the main screen.
- Design the UI using Jetpack Compose.
- Display the title "Lab 3 - App Navigation".
- Add a button labeled "Go to Second page".
- Create another activity called SecondActivity.
- When the button is clicked, use Intent to navigate to SecondActivity.
- Display the content of the second screen.
- Run the application and verify navigation between screens.
- Stop the application.

- Add a circle button.
- When the circle button is pressed:
  - Update the state variable to display a circle shape in the drawing area.
- Add a Show All button.
- Add a Clear All button.
- Use compose recomposition to update the UI whenever the state changes.
- Verify that the shapes appear and disappear correctly based on button clicks.
- Stop the application.

Result:

The Android application was successfully developed using Kotlin and Jetpack compose, allowing the user to draw and control shapes such as lines and circles using layout components and buttons.



04/02/26

Experiment - 4

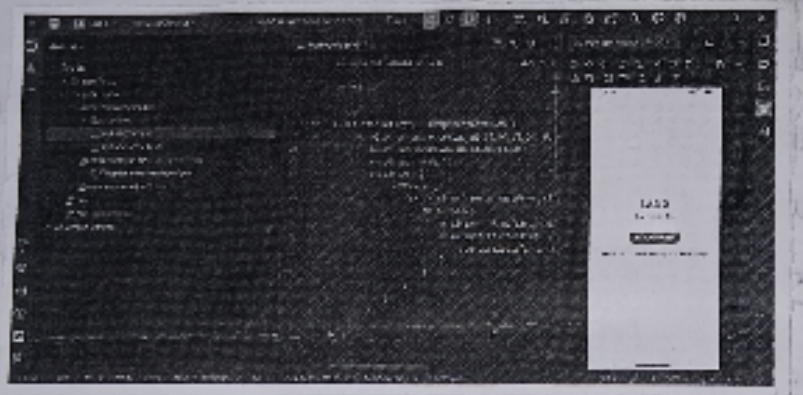
Timer Application with Activity Lifestyle

AIM:

To develop an Android application using Kotlin that implements a stopwatch timer with start, Pause, and stop controls and demonstrates Android Activity Lifecycle events.

ALGORITHM:

- Start the application.
- Create a new Android project in Android Studio using Kotlin.
- Design the layout with:
  - A TextView to display the time.
  - three Buttons : Start, Pause and stop
  - A TextView to display lifecycle logs.
- Initialize variables for startTime, elapsed time, and timer state.
- Use a Handler and Runnable to update the timer continously.
- When the Start Button is clicked:
  - Record the start time using SystemClock.elapsedRealtime().
- Implement Activity lifecycle methods.
- log each lifecycle event and display it in the Lifecycle Log Textview.
- Run the application on the Android emulator or device.



Result :

The Android application successfully demonstrates navigation from the main screen to a second screen using a button click.

06/02/26

Experiment - 5

Implement UI components using Jetpack Compose.

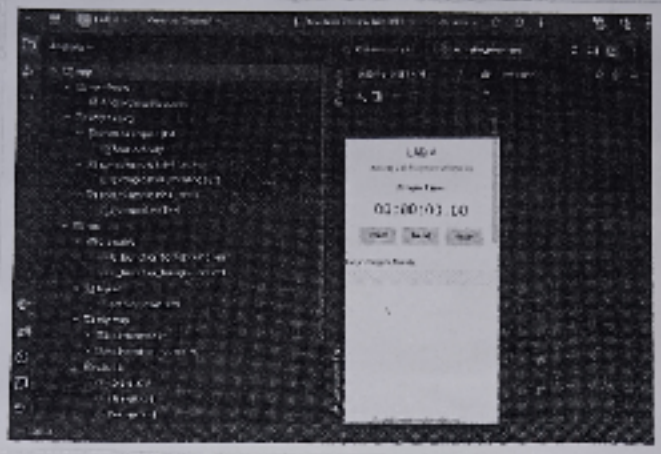
Aim:

To develop an Android application using Kotlin and Jetpack Compose that demonstrates the UI layer components such as buttons, cards, rounded boxes and colored elements.

ALGORITHM:

- Start the application.
- Open Android studio and create a new project using Kotlin.
- Implement the MainActivity class.
- Use Jetpack compose to design the user interface. Create a MainScreen composable function to display sample information.
- Add a Rounded Box using compose UI modifiers.
- Display colored shapes (Red, Green, Blue) using Box components. Arrange all components using column and Row layouts.
- Run the application on an Android emulator or device.
- verify that all UI components are displayed correctly.
- Stop the application.

- Stop the application



Result:

The Android application successfully implements a stopwatch timer with control buttons and demonstrates Android Activity Lifecycle logging.

11/02/26 Experiment-6

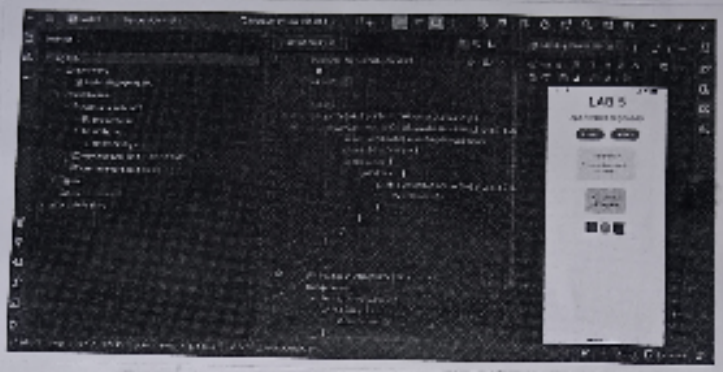
App Architecture - Persistence

AIM:

To develop an Android application using Kotlin and Jetpack compose that demonstrates data persistence techniques such as SharedPreferences and Internal Storage.

ALGORITHM:

- Start the application.
- Create a new Android project in Android Studio using Kotlin. Implement the MainActivity class.
- Use Jetpack compose to design the user interface. Display the title "LAB 6 - App Architecture: Persistence".
- Create a section to explain architecture components like: SharedPreferences, Internal Storage and Room Database.
- Implement a SharedPreferences section with:
  - A textfield to enter a user name.
  - A SaveName button to store the name.
- Use SharedPreferences API to store and retrieve the entered data.
- Read and display the stored note when required. Arrange the UI Components using column, Card, and TextField layouts.
- Run the application on the Android emulator or device.
- Stop the application



Result:

The Android application successfully demonstrates the implementation of Jetpack Compose UI components such as buttons, cards, boxes and layout structures.

20/02/26 Experiment-7

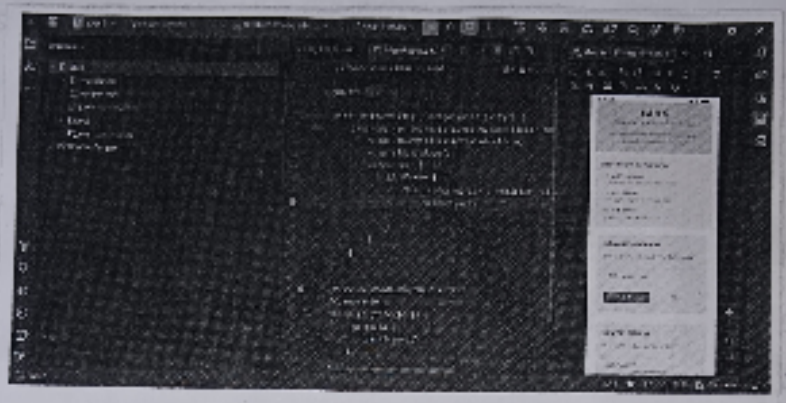
Advanced RecyclerView Implementation.

Aim:

To develop an Android application using Kotlin that demonstrates the use of Recyclerview to display a list of items efficiently with advanced features.

Algorithm:

- Start the application.
- Create a new Android project in Android Studio using Kotlin.
- Add a Recyclerview component in the activity layout. Create a data model class to represent the list items.
- Create an Adapter class to bind data to the Recyclerview items.
- Implement a ViewHolder class inside the adapter. Initialize the Recyclerview in MainActivity.
- Create a sample list of data items.
- connect the adapter with the Recyclerview.
- Implement item interactions such as click events or updates.
- Run the application on the Android emulator or device.
- verify that the list items are displayed and scroll correctly.
- Stop the application.



Result:

The Android Application Successfully demonstrates data persistence techniques.

25/02/26 Experiment-8

Connect to the Internet (Network connectivity).

AIM:

To develop an Android application using Kotlin and Jetpack Compose that checks and displays the internet connectivity status of the device.

ALGORITHM:

- Start the application.
- Create a new Android project in Android Studio using Kotlin.
- Implement the MainActivity class.
- Add the required internet permission in the AndroidManifest.xml.
- Use ConnectivityManager and NetworkCallback to monitor network status.
- Create a composable function to display the connection status on the screen.
- Observe the network state changes using a connectivity observer.
- If the device is connected to the internet, display the message "Connected" with a green indicator.
- Use Jetpack Compose UI components such as column, card, Text and Icon to design the interface.
- Run the application on the Android emulator or a physical device.
- Stop the application.

Result :

The Android application successfully demonstrates the implementation of RecyclerView.

12/03/26 Experiment-9.

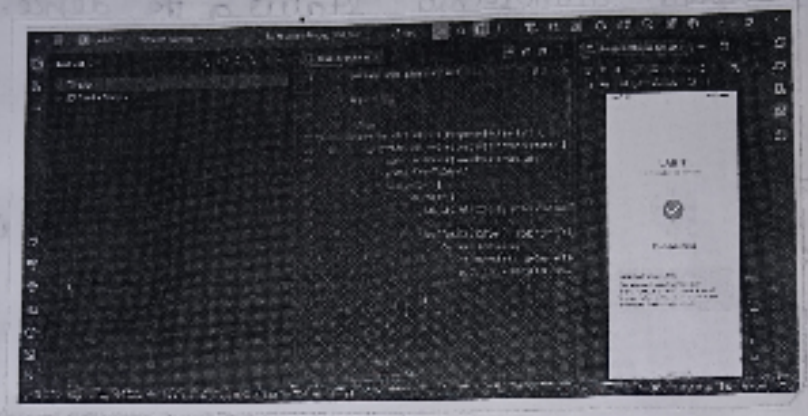
Repository Pattern and WorkManager.

AIM:

To develop an Android application using Kotlin and Jetpack compose that demonstrates the Repository Pattern and WorkManager for background task execution and data synchronization

ALGORITHM:

- Start the application.
- Create a new Android project in Android studio using Kotlin.
- Implement the MainActivity class.
- Design the user interface using Jetpack Compose.
- Display the title "LAB-9. Repository Pattern and WorkManager".
- Implement a Repository class to manage and provide data to the UI layer.
- Create a Goal data model to represent items displayed on the screen.
- Use WorkManager to schedule background tasks. Create a WorkerClass that performs background synchronization.
- Display the synchronization status on the screen.
- Verify that the background task executes correctly. Stop the application.



Result:

The Android application successfully checks and displays the internet connectivity status of the device using Connectivity Manager and Jetpack Compose UI.

26/03/26. Experiment-10:

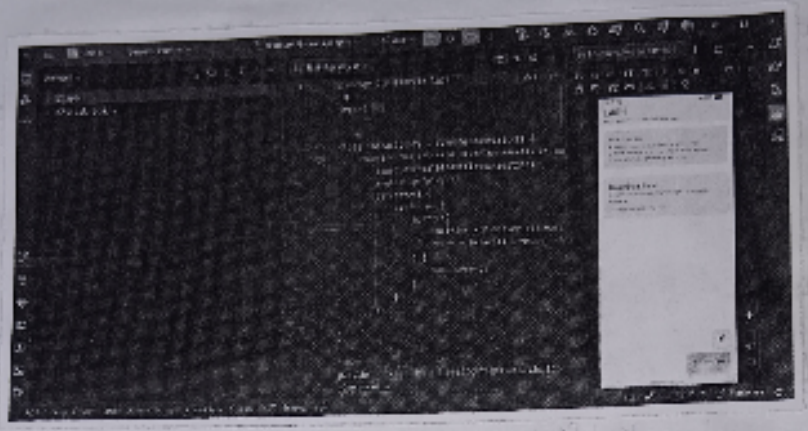
App UI Design using Navigation

Aim:

To develop an Android application using Kotlin and Jetpack Compose that demonstrates UI design principles and navigation between screens.

Algorithm:

- Start the application
- Create a new Android project in Android Studio using Kotlin. Implement the MainActivity class.
- Use Jetpack compose to design the user interface. Implement Navigation component for screen navigation.
- Create different composable screens such as Home screen and other UI concept screens.
- Display the title "LAB 10 - App UI design" on the home screen.
- When a user selects an item, navigate to the corresponding screen using NavControllers.
- Apply proper layout, spacing and UI styling using Compose components.
- Run the application on the Android emulator or device.
- Verify the UI elements and navigation work correctly.
- Stop the application.



Result:

The Android application successfully demonstrates the use of the Repository Pattern and WorkManager for handling background tasks.